Stop hand-tuning kernels: How Neuron Agentic Development accelerates AWS Trainium optimizations

by Josh Longenecker and John Liu | on 10 JUN 2026 | in Advanced (300), Amazon EC2, Technical How-to | Permalink | Share

As frontier AI models grow in scale and complexity, developers face a common challenge across every hardware platform: how do you extract the maximum performance and efficiency from the silicon their models run on. Whether delivering real-time experiences for world models, supporting deeper reasoning in agentic workflows, or reducing inference costs at scale, the gap between what hardware can theoretically deliver and what most teams achieve remains significant. Custom kernel development has historically been the path to closing that gap, but it demands deep architectural expertise, manual profiling workflows, and iterative optimization cycles that few teams can afford.

This doesn't need to be the case. What if every machine learning (ML) engineer could operate as a performance engineer, writing hardware-aware kernels, diagnosing bottlenecks, and shipping optimized models, without years of chip-level experience? What if developers already proficient on one architecture could ramp up on another in days instead of months?

Today, we're announcing the Neuron Agentic Development capabilities: a collection of AI agents and skills that make this possible for developers building on AWS Trainium and AWS Inferentia. The first capabilities equip coding agents in Kiro and Claude to author, debug, and profile Neuron Kernel Interface (NKI) kernels, extending ML performance engineering to every developer on the team. Kernel developers coming from other architectures can scale quickly to Trainium, teams can shorten the time from idea to hardware-optimized implementation, and the deep architectural knowledge that once gatekept kernel development is now accessible through agentic tooling that guides developers at each step.

In this post, we explain how the Neuron Agentic Development capabilities accelerate the kernel development workflow.

The Neuron Agentic Development skills

The Neuron Agentic Development package provides five specialized skills that follow the natural kernel development pipeline: **write** → **debug** → **profile** → **analyze**. You can invoke skills individually for targeted tasks, or chain them together with the `neuron-nki-agent`, which auto-selects the right workflow based on your request. To use them, add the skills to your agentic IDE's skills directory. For example, in any IDE like VS Code, Cursor, or Kiro, add the skills in the `.kiro/skills` or `.claude/skills` directory and make them available to your agents. Skills must run on a Trainium-based Amazon Elastic Compute Cloud (Amazon EC2) instance.

Kernel authoring

The `neuron-nki-writing` skill is your starting point for creating NKI kernels. It translates PyTorch, NumPy, or natural language descriptions into correct NKI code. For example, it covers tiling strategies that respect hardware constraints (such as 128 partition dimension and 512/4096 PSUM free dimension), memory access patterns, compute operations with explicit `dst` parameters, and efficiency guidelines for DMA sizing and SBUF reuse. The skill classifies your task by complexity and loads only the references needed.

Debugging

The `neuron-nki-debugging` skill provides a systematic workflow for resolving NKI compilation and execution errors on Trainium and Inferentia hardware. For example, it covers environment setup with the correct `--target` flags, compiler error

resolution with a categorized index of all 28 NCC error codes, and numerical validation against CPU-computed references.

Profiling and analysis

The `neuron-nki-profiling` skill captures execution profiles on hardware. It configures runtime inspection environment variables, runs the kernel, identifies the correct Neuron Execution File Format (NEFF), and captures the trace with `neuron-explorer` including DGE (DMA Graph Engine) notifications for DMA-level detail. It extracts JSON metrics and produces the NEFF files that `neuron-nki-profile-querying` consumes.

The `neuron-nki-profile-querying` skill ingests NEFF and NTFF files and runs SQL queries to compute performance bounds, identify bottleneck engines, and localize inefficiencies to specific NKI source lines. It supports three analysis approaches: the `neuron-explorer` API server, DuckDB directly on parquet, or pandas for custom computation.

Documentation

The `neuron-nki-docs` skill is used throughout development. During authoring, it provides API signatures and tutorials. During debugging, it explains error codes. During profiling, it clarifies hardware architecture details. Ask about a specific `nisa.*` or `nl.*` API, look up error codes, find tutorials, or browse architecture guides for Trainium 1, 2, and 3.

The agents

While skills provide building blocks for individual tasks, agents combine multiple skills into autonomous workflows. Each agent is a specialized persona that handles multi-step development scenarios end-to-end.

- The `neuron-nki-agent` is the unified entry point for NKI development. It automatically selects the right workflow based on your request (writing, debugging, profiling, or documentation lookup) and orchestrates the appropriate skills. This is the default starting point.
- The `neuron-nki-writing-agent` focuses exclusively on kernel authoring. It translates PyTorch, NumPy, or natural language descriptions into NKI code and handles modifications to existing kernels.
- The `neuron-nki-debugging-agent` autonomously resolves compiler errors by analyzing the error, searching documentation for fixes, and applying corrections. It tracks iterations (up to 10) and progressively simplifies when stuck.
- The `neuron-nki-docs-agent` is a lightweight documentation navigator for API signatures, error code explanations, tutorials, and architecture details.
- The `neuron-nki-profile-analysis-agent` runs two separate skills to identify performance bottlenecks. It uses the `neuron-nki-profile` skill to capture execution profiles on hardware: it sets environment variables, runs the kernel, identifies NEFFs, and runs `neuron-explorer` capture to produce profile parquet files. It then uses the `neuron-nki-profile-querying` skill to run SQL queries against those parquet files to compute performance bounds, identify bottleneck engines, and localize inefficiencies to specific NKI source lines.

Putting it into practice: Optimizing a custom softmax kernel

The following walkthrough shows how these agentic capabilities work together in practice. You explore two kernels: a softmax kernel (Steps 1 and 2) and a SwiGLU kernel (Steps 3 and 4), which demonstrates profiling on a real-world workload.

Suppose you have a PyTorch softmax operation that's a bottleneck in your inference pipeline, and you want to write a custom NKI kernel to fuse it with a preceding scale operation.

Step 0: Set up your instance and environment

To get up and running:

1. Launch a `trn2.3xlarge` instance through AWS MLCBs using the AWS Neuron Deep Learning AMI (DLAMI). São Paulo (sa-east-1) and Melbourne (ap-southeast-4) are used as example AWS Regions here. See the full Trainium availability list for other supported Regions.
2. Connect by using SSH into the instance.
3. Install Kiro:

```
curl -fsSL https://cli.kiro.dev/install | bash
```

4. Install Neuron Agentic Development skills following the instructions at the neuron-agentic-development repository.

Note: `trn2.3xlarge` instances incur hourly charges while running. Remember to terminate the instance when you finish this walkthrough to avoid ongoing costs.

For more detailed instance setup and configuration instructions, see the Neuron DLAMI Setup Guide.

From the remote terminal, verify the neuron devices are visible:

```
# Confirm Neuron devices are visible
neuron-ls

# Confirm neuron-explorer is available
which neuron-explorer && neuron-explorer --version
```

The DLAMI comes with a pre-installed virtual environment at:

```
~opt/aws_neuronx_venv_pytorch_2_9
```

Activate it with:

```
source ~opt/aws_neuronx_venv_pytorch_2_9/bin/activate
```

With the environment setup, you can get started developing kernels by running:

```
kiro-cli --agent neuron-nki-agent
```

Step 1: Write the kernel

In the interactive Kiro CLI session, enter the following prompt: “Write an NKI kernel that computes scaled softmax: softmax(x * scale) along the last dimension, for input shape [batch, seq_len, hidden_dim] in bfloat16.”

The agent produces a complete three-pass kernel (row max, sum-of-exp, normalize) using `nisa.activation(np.exp, ...)` for hardware-accelerated exp, float32 accumulation for numerical stability, and proper tiling across the free dimension. It explains its design decisions: one program instance per row, P_MAX=128 (matching the 128-partition hardware limit), F_MAX=2048 (matching the 2048-element free dimension limit on Trainium), and bfloat16 output cast.

```
❯ kiro-cli --agent neuron-nki-agent
  Now I have all the information needed. Let me mark the first task complete and write the kernel.
● Tasks updated

  Now I'll write the kernel. The strategy:

  - Reshape [B, S, H] → [B*S, H] so softmax along H is a reduction along the free dimension (axis=1)
  - Tile the partition dimension (B*S) in chunks of 128
  - For each tile: scale → max → subtract max → exp → sum → reciprocal → multiply
  - Use fused nisa.activation(op=nl.exp, data=x, scale=scale) for the initial scaling + exp step? Actually no — we need to subtract max first. So: scale → max → subtract → exp → sum →
  reciprocal → multiply.

  Wait, let me reconsider. The operation is softmax(x * scale). The numerically stable version is:

  1. scaled = x * scale
  2. m = max(scaled, axis=-1)
  3. e = exp(scaled - m)
  4. s = sum(e, axis=-1)
  5. result = e / s

  I can fuse step 1 into step 3 using nisa.activation(op=nl.exp, data=x, scale=scale, bias=-m) — but bias needs to be a tensor matching the partition dim. Let me check if that's
  possible... Actually the fused pattern is exp(x * scale + bias) which would be exp(x * scale - max). But max is per-row, so I need to compute max first from x * scale.

  Simpler approach: compute x * scale first, then do standard softmax on that.
● Write /Users/jlongen/Development/neuron-agentic-development/scaled_softmax_nki.py
     1   # Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
     2   # SPDX-License-Identifier: Apache-2.0
     3
     4   """
     5   Scaled softmax NKI kernel: softmax(x * scale) along the last dimension.
     6
     7   Input shape: [batch, seq_len, hidden_dim] in bfloat16.
     8   Computes numerically stable softmax: exp(x*scale - max(x*scale)) / sum(exp(x*scale - max(x*scale)))
     9   """
    10
    11   import nki
    12   import nki.isa as nisa
    13   import nki.language as nl
    14
    15
    16   P_MAX = 128
    17
    18
    19   def div_ceil(n: int, d: int) -> int:
    20       return (n + d - 1) // d
    21
    22
    23   @nki.jit
    24   def scaled_softmax_kernel(input_tensor, scale=1.0):
    25       """
    26       Compute softmax(input_tensor * scale) along the last dimension.
    27
    28       Args:
    29           input_tensor: [batch, seq_len, hidden_dim] @ HBM, bfloat16
    30           scale: scalar multiplier applied before softmax
    31
    32       Returns:
    33           [batch, seq_len, hidden_dim] @ HBM, bfloat16
    34       """
    35       batch, seq_len, hidden_dim = input_tensor.shape
0 ⚠ 0
```

Figure 1: NKI agent authoring a kernel.

Step 2: Debug on hardware

Ask the agent to run the kernel and verify numerical parity against a PyTorch reference.

The agent hits an immediate snag: `nisa.tensor_tensor` doesn't auto-broadcast reduction results, so the per-row max and sum values can't be directly applied across the full hidden dimension. The agent consults the NKI reference patterns, identifies the correct broadcast mechanism (stride-0 access views via `.ap()`), and rewrites the kernel accordingly.

After syncing the corrected kernel to the instance and running on-device:

```
PASS: shape=(2, 128, 512), max_diff=0.000008
PASS: shape=(4, 256, 1024), max_diff=0.000004
PASS: shape=(1, 1, 64), max_diff=0.000061
PASS: shape=(2, 300, 768), max_diff=0.000007

All tests passed.
```

All four cases pass with max error well within bfloat16 tolerance, confirming the kernel is numerically correct on real Trainium hardware.

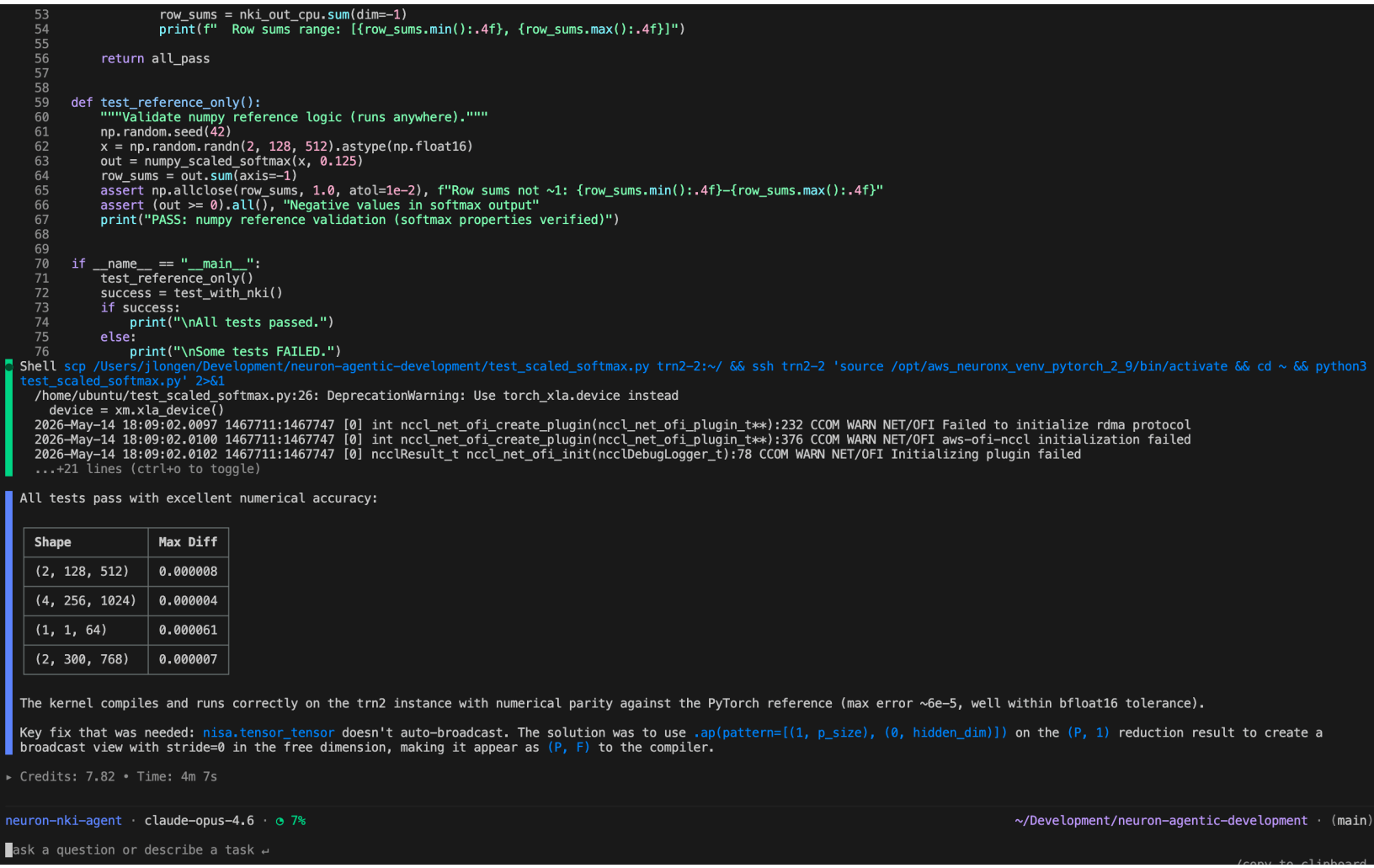


Figure 2: NKI agent debugging its mistakes.

Step 3: Profile and analyze kernel execution

After the kernel compiles and produces numerically correct results, the next step is to profile execution on hardware to identify performance bottlenecks and guide optimizations.

To demonstrate profiling and analysis on a real-world workload, this step uses a SwiGLU MLP kernel, a common module in large language models (LLMs).

Point the agent at the SwiGLU kernel and ask it to analyze the profile. The agent first compiles the kernel to a NEFF and captures an NTFF trace through `neuron-explorer`. Then it runs a two-part investigation into the kernel, looking first at kernel-level statistics and performance bounds, and then deep into specific inefficiencies by querying the profile at the instruction execution level.

First the agent runs a full bounds analysis on the captured profile and finds multiple gaps worth investigating:

All investigations are complete. Let me mark the task and present the final report.
● Tasks updated

SwiGLU v6 NKI04 — Profile Analysis Report

Kernel: swiglu_mlp_kernel (4096 tokens × 4096 input × 8192 hidden, fp16)
Total execution time: 15,486 µs
Bottleneck engine: Tensor Engine (12,431 µs, 80.3% of total)

Bounds Table

Bound	Time (µs)	Gap to next	% of total
total_time	15,486		
perfect_pipeline	12,431	3,056 µs (pipeline gap)	19.7%

Engine active times:

Engine	Time (µs)	% of total
Tensor (bottleneck)	12,431	80.3%
DMA	9,686	62.5%
Vector	4,846	31.3%
Scalar	323	2.1%

Memory family:

Bound	Time (µs)	Gap	% of total
memory_bound	9,686		
memory_bound_ideal	3,857	5,830 µs (efficiency)	37.6%
memory_bound_no_reloads	617	3,240 µs (excess traffic)	20.9%

Compute family:

Bound	Time (µs)	Gap	% of total
compute_bound (TE active)	12,431		
compute_bound_ideal_flops	10,813	1,617 µs (TE efficiency)	10.4%
compute_bound_ideal_useful	10,486	328 µs (transposes)	2.1%

Key derived metrics:

- Achieved DMA BW: 173 GB/s (peak 435 GB/s)
- Achieved TE: 68.4 TFLOPS (87% of 78.6 peak)
- DMA excess ratio: 6.2×

Figure 3: NKI agent extracts summary statistics and calculates performance bounds on the kernel.

It finds multiple gaps worth investigating further. The TE engine dominates execution and is inefficient. It also has large idle gaps, which suggests it might be worth investigating its most likely dependency (DMA engine), where we can see work that is both redundant and inefficient.

Investigation Findings

Ordered by relevance to the bottleneck (Tensor Engine):

1. TE Efficiency Gap — 1,617 µs (10.4%)

96.7% of hardware FLOPs execute at peak tile size (K=128, M=128, N=512). The remaining 3% use N=128 (undersized free dimension):

Source	Tiles	% of flops	Tile dims	Issue
kernel.py:258 (`nc_transpose` of hidden_sub)	2,048	2.0%	128×128×128	N=128
kernel.py:110 (`nc_transpose` of x_tile)	1,024	1.0%	128×128×128	N=128

The undersized tiles are all transposes (N=128 vs 512 max). The remaining 1,617 µs gap beyond what tile sizes explain is likely from instruction startup overhead and potential data starvation effects.

2. TE Transpose Gap — 328 µs (2.1%)

3,072 TRANSPOSE MATMULs consume 25.77 GFLOPS (3% of total hw_flops):

Source	Count	GFLOPS	Gap (µs)	% of transpose
kernel.py:258 (`nc_transpose` of `hidden_sub_f32`)	2,048	17.18	219	67%
kernel.py:110 (`nc_transpose` of `x_tile_f32`)	1,024	8.59	109	33%

These are required for the matmul layout (transposing activations to stationary format).

3. Pipeline Gap — 3,056 µs (19.7%)

The gap between total*time and perfect*pipeline indicates engines are not fully overlapped — there is serialization where TE is idle waiting for data or synchronization.

4. DMA Efficiency Gap — 5,830 µs (37.6% of total)

DMA achieves only 173 GB/s of 435 GB/s peak. All weight loads use descriptor size 1,024B — well below the 4 KB threshold for efficient transfers:

Source	Tensor	Dir	Xfers	MB	%	Shape
kernel.py:161	gate_weight	load	4,096	536	32.0%	128×1024B
kernel.py:153	up_weight	load	4,096	536	32.0%	128×1024B
kernel.py:182	down_weight	load	4,096	530	31.6%	128×1024B
kernel.py:95	x (input)	load	1,024	47	2.8%	128×256B

Since DMA is not the bottleneck, this gap doesn't directly limit performance — but it contributes to the pipeline gap by making TE wait for data.

5. DMA Excess Traffic — 3,240 µs at ideal BW (20.9%)

6.2× excess ratio. 100% from input reloads, 0% from spills. Each weight tensor (input0/1/2 = gate/up/down weights, 67 MB each) is loaded 8× from HBM:

Source	Tensor	Xfers	MB
kernel.py:182	down_weight	3,382	443
kernel.py:161	gate_weight	3,381	443
kernel.py:153	up_weight	3,377	443

The 8× reload factor comes from the kernel's loop structure: weights are reloaded for each of the 4 token tiles in the batch × 2 (since num_batches = tokens/BATCH7/FSTAT_MAX). The weights don't fit in SBUF alongside activations.

Summary

The kernel is **Tensor Engine bound** at 87% TE utilization. The largest optimization opportunities are:

1. **Pipeline gap (19.7%)** — TE is idle for 3,056 µs waiting on other engines. Improving DMA efficiency or reducing weight reloads could feed TE faster and shrink this gap.
2. **TE efficiency (10.4%)** — mostly explained by transpose tile sizes (N=128) and potential data starvation effects.
3. **TE transposes (2.1%)** — 3,072 transposes at kernel.py:258 and :110 are structurally required for the matmul layout.

The DMA side has large absolute gaps (efficiency + excess traffic) but since DMA is not the bottleneck, these only matter insofar as they contribute to the pipeline gap starving TE.

Figure 4: NKI agent investigates inefficiencies in the profile and provides an analysis.

The investigations help us audit the gaps and prioritize actionable optimization directions. While the bottleneck engine's (Tensor Engine) inefficiency would have been the top target for optimization, the agent finds that the NKI matmul instructions are already performing near their peak efficiency. In contrast, we find that DMA instructions are well below their target size (inefficient) and that we are also reloading all inputs eight times (redundant). We even find the three exact lines of NKI code responsible for the suboptimal transfers. Addressing these lines might in turn reduce the TE's idle gap and improve kernel latency.

Things to know

Keep the following considerations in mind when working with Neuron Agentic Development skills and agents.

- Profiling and debugging skills require execution on actual Trainium or Inferentia-based instances.
- The writing and docs skills work anywhere.
- All skills target the current NKI API (0.4.0 at time of publication). Skills support Trainium1 (gen2), Trainium2 (gen3), and Trainium 3 (gen4) with appropriate `--target` flags.
- The skills and agents are designed to work together. The top-level agent automatically invokes profiling and debugging skills as needed.

Cleanup

To avoid ongoing charges, terminate the `trn2.3xlarge` instance you created in Step 0. You can do this through the AWS Management Console (**EC2 > Instances**, select the instance, and choose **Instance state > Terminate**), or run:

```
aws ec2 terminate-instances --instance-ids <your-instance-id>
```

Confirm that the instance state shows "terminated" before closing the console.

What's next

The kernel authoring and profiling skills lower the barrier to writing high-performance kernels on Trainium, but they are only the first part of a broader vision.

Today, developers use profiling insights to guide their next round of kernel edits. This iterative cycle (profile, diagnose, refactor, re-profile) is where the most time is spent. We want to make this loop fully agentic. For example, agents that autonomously iterate on a kernel until it meets its performance target, without requiring the developer to interpret each profiling result and hand-craft the next fix.

We also hear from performance developers that custom kernels are only one part of a larger challenge. Developers want their models to run on Trainium without having to worry about porting model code and syntax, resolving operator gaps, applying model-level optimizations, and validating correctness at scale. We want to bring the same agentic approach to this broader problem.

In summary, our vision is to support the next wave of innovations for frontier models using Trainium and the Neuron SDK, and to use the suite of Neuron Agentic Development capabilities to achieve leading cost-performance for use cases ranging from experimentation with new model architectures to running production models at scale.

We will share more as these capabilities mature. To get started with what's available today, visit the Neuron Agentic Development GitHub repository.

Come build with us

The Neuron Agentic Development capabilities are available today. Get started now: clone the neuron-agentic-development repository and write your first NKI kernel in minutes.

Here's how to dive in:

1. Start with the `neuron-nki-agent`. It selects the right workflow based on your request, giving you the full autonomous experience end-to-end.
2. Run the skill examples. Invoke individual skills directly (for example, `/neuron-nki-writing`) for targeted tasks, or chain `/neuron-nki-profiling` and `/neuron-nki-profile-querying` once your kernel is producing correct results.
3. Open a GitHub issue if you run into a problem or have an idea. We're actively developing alongside the community and will get back to you.
4. Contribute back. Submit PRs, share kernels you've built, and help us make these tools better for everyone.

We're building these capabilities in the open because the best developer tools are shaped by the developers who use them. Come build with us.